

AI ASSISTED CODING

LAB ASSIGNMENT –10.4

SAHASRA VEERAGONI

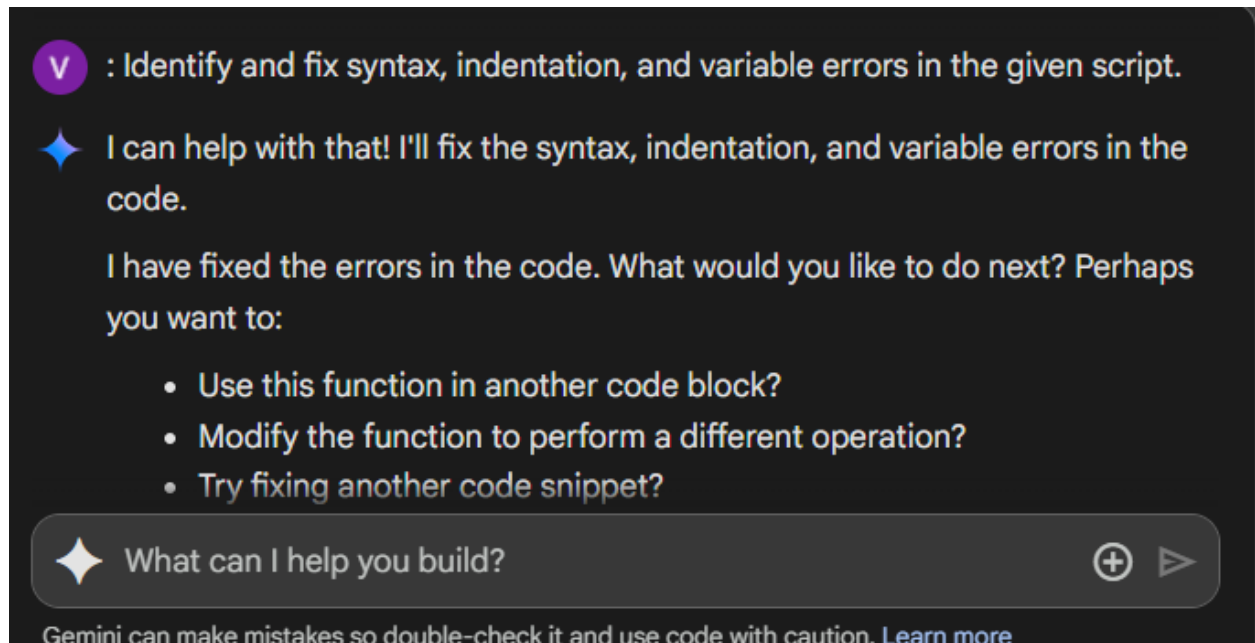
2403A53007

24BTCAICYB01-2-1

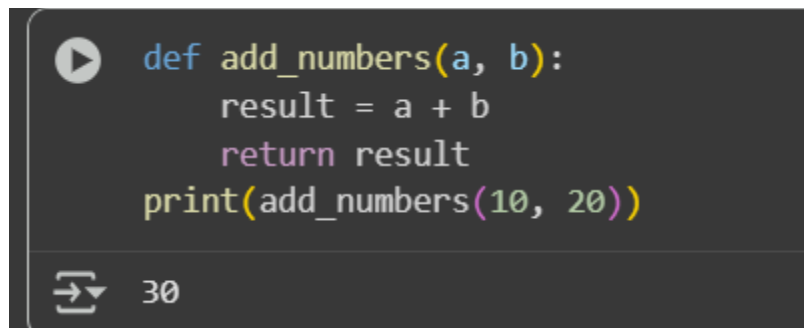
#TASK-1

Identify and fix syntax, indentation, and variable errors in the given script.

Prompt:



OUTPUT:



#TASK-2

Optimize inefficient logic while keeping the result correct.

PROMPT:

V "Write a Python function that scans a list of integers and returns a new list containing only the numbers that appear more than once, without repeating any duplicates in the output."

✦ Okay, I can help you with that. I will update the `find_duplicates` function to efficiently find and return unique duplicates from the list.

I have updated the `find_duplicates` function to return a list of numbers that appear more than once, without repeating any duplicates in the output.

OUTPUT:

```
from collections import Counter

def find_duplicates(nums):
    counts = Counter(nums)
    duplicates = [num for num, count in counts.items() if count > 1]
    return duplicates

numbers = [1, 2, 3, 2, 4, 5, 1, 6, 1, 2]
print(find_duplicates(numbers))
```

[1, 2]

#TASK-3

Refactor messy code into clean, PEP 8-compliant, well-structured code

PROMPT:

 Write a Python function named `calculate_factorial` that computes the factorial of a given non-negative integer. Ensure the code has proper indentation, meaningful variable names, docstrings explaining the function, and is formatted for readability."

 I can certainly help you with that! I will modify the existing cell to create a well-formatted `calculate_factorial` function with a docstring.

I have created the `calculate_factorial` function in the cell above, ensuring it has proper formatting, meaningful variable names, and a docstring as you requested.

OUTPUT:

```
def calculate_factorial(n):  
    """  
    Computes the factorial of a non-negative integer.  
  
    Args:  
        n: A non-negative integer.  
  
    Returns:  
        The factorial of n.  
    """  
    if n == 0:  
        return 1  
    else:  
        factorial_result = 1  
        for i in range(1, n + 1):  
            factorial_result *= i  
        return factorial_result  
  
# Example usage:  
print(calculate_factorial(5))
```

120

#TASK-4

Add security practices and exception handling to the code.

PROMPT:

V "Refactor the given Python code that fetches user data from an SQLite database to include proper security practices and exception handling. Ensure the code prevents SQL injection, safely handles database connections, and gracefully manages errors. Maintain readability and include comments explaining the changes."

✦ I understand! I will refactor the code in the cell above to address the SQL injection vulnerability and include proper error handling for database operations.

OUTPUT:

```
import sqlite3

def get_user_data(user_id):
    """
    Fetches user data from the users.db database for a given user ID.

    Args:
        user_id: The ID of the user to fetch data for.

    Returns:
        A list of rows matching the user ID, or an empty list if an error occurs
        or no data is found.
    """
    conn = None # Initialize connection to None
    try:
        conn = sqlite3.connect("users.db")
        cursor = conn.cursor()

        # Use parameterized query to prevent SQL injection
        query = "SELECT * FROM users WHERE id = ?;"
        cursor.execute(query, (user_id,))

        result = cursor.fetchall()
        return result

    except sqlite3.Error as e:
        print(f"Database error: {e}")
        return [] # Return empty list on error

    except Exception as e:
        print(f"An unexpected error occurred: {e}")
        return [] # Return empty list on unexpected error
```

```

# Use parameterized query to prevent SQL Injection
query = "SELECT * FROM users WHERE id = ?;"
cursor.execute(query, (user_id,))

result = cursor.fetchall()
return result

except sqlite3.Error as e:
    print(f"Database error: {e}")
    return [] # Return empty list on error

except Exception as e:
    print(f"An unexpected error occurred: {e}")
    return [] # Return empty list on unexpected error

finally:
    # Ensure the connection is closed even if errors occur
    if conn:
        conn.close()

# Example usage:
# Note: In a real application, you would not take user input directly like this
# due to potential security risks even with parameterized queries if not
# properly validated. This is for demonstration of the function itself.
user_input = input("Enter user ID: ")
print(get_user_data(user_input))

Enter user ID: SAHASRA
Database error: no such table: users
[]

```

#TASK-5

Generate a review report for this messy code

PROMPT:

 GIVE ME A PROPER CODE WITH FILLING Missing docstrings,Inconsistent formatting (indentation, inline return),Missing error handling for division by zero,Non-descriptive function/variable names AND Suggestions for readability and PEP 8 compliance

 I can help you with that! I will refactor the code in the cell above to address the missing docstrings, inconsistent formatting, missing error handling for division by zero, and non-descriptive names, while also improving readability and PEP 8 compliance.

OUTPUT:

```

def perform_calculation(num1, num2, operation):
    """
    Performs a basic arithmetic calculation based on the specified operation.

    Args:
        num1: The first number.
        num2: The second number.
        operation: A string representing the operation to perform ('add', 'sub', 'mul', 'div').

    Returns:
        The result of the calculation, or None if the operation is invalid or
        division by zero occurs.
    """
    if operation == "add":
        return num1 + num2
    elif operation == "sub":
        return num1 - num2
    elif operation == "mul":
        return num1 * num2
    elif operation == "div":
        if num2 == 0:
            print("Error: Division by zero is not allowed.")
            return None
        return num1 / num2
    else:
        print("Error: Invalid operation.")
        return None

# Example usage:
print(perform_calculation(10, 5, "add"))
print(perform_calculation(10, 0, "div"))
print(perform_calculation(10, 5, "sub"))

```

```

# Example usage:
print(perform_calculation(10, 5, "add"))
print(perform_calculation(10, 0, "div"))
print(perform_calculation(10, 5, "sub"))
print(perform_calculation(10, 5, "mul"))
print(perform_calculation(10, 5, "mod"))

```

```

15
Error: Division by zero is not allowed.
None
5
50
Error: Invalid operation.
None

```